HADOOP vs SPARK

T120M337 Distributed Systems and Algorithms

Performance Analysis of Distributed Computing Frameworks for Big Data Analytics: Hadoop vs Spark

Shwet Ketu · Pramod Kumar Mishra · Sonali Agarwal
Computación y Sistemas 24(2), 2020 · DOI: 10.13053/CyS-24-2-3401

Student: Alikhan Sagymbekov
Group: IFMU-5
Supervisor: Assoc. Prof. Germanas Budnikas

Scientific paper analysis + two adapted experiments

Paper scope and selected reproduction

- The paper compares execution models, capabilities, and performance.
- Original experiments: WordCount, logistic regression, and K-Means.
- Selected here: WordCount input-size scaling and iterative K-Means.
- Real Hadoop Streaming and PySpark; one computer with HDFS/YARN.
- Adapted datasets and sizes; no claim of exact numerical replication.

Ketu et al. (2020), §§4–5; own reproduction scope

Research questions for this reproduction

- RQ1: How do Hadoop MapReduce and Spark execute the same task?
- RQ2: How does WordCount time change with input size?
- RQ3: How does K-Means time change with iteration count?
- RQ4: Do the computed outputs and centers agree?
- RQ5: Which settings and implementation differences limit our conclusions?

Questions formulated for this project, not verbatim questions from the paper

Framework fundamentals

- HDFS stores inputs and outputs; YARN allocates task resources.
- Hadoop: Python mapper → Hadoop shuffle/sort → Python reducer.
- Spark: RDD transformations form a plan; actions execute it.
- Hadoop K-Means launches a new job for every iteration.
- Spark K-Means reuses one application and a persisted RDD; spilling is possible.

Hadoop/Spark documentation; own implementation

Original experimental design

- WordCount: Wikilinks and Wikilanguage; figure sizes 500 MB–40 GB.
- K-Means: four 5 GB corpora at 1–3 iterations; a 40 GB corpus at up to 9.
- Figure 8: WordCount advantage narrows with size (102/54 → 1762/1630 s).
- Figure 16: iterative advantage grows; first in-memory iteration is slower.
- Our data, feature pipeline, versions, and scale differ from the paper.

Ketu et al. (2020), Fig. 8–9 and 14–16; values shown as Hadoop/Spark

Own environment and controlled resource requests

- Intel Core i5-10300H; ~23.3 GiB usable RAM; Linux; one node.
- Python 3.12.3 · Java 17 · Hadoop 3.5.0 · Spark 3.5.7.
- Workers: up to 2 vcores and 2048 MiB requested by each engine.
- Hadoop: up to 2 tasks × 1024 MiB; Spark: one two-core executor.
- Same HDFS inputs; 3 repetitions; recorded configurations and checksums.
- Worker requests are comparable; total process resources are not identical.

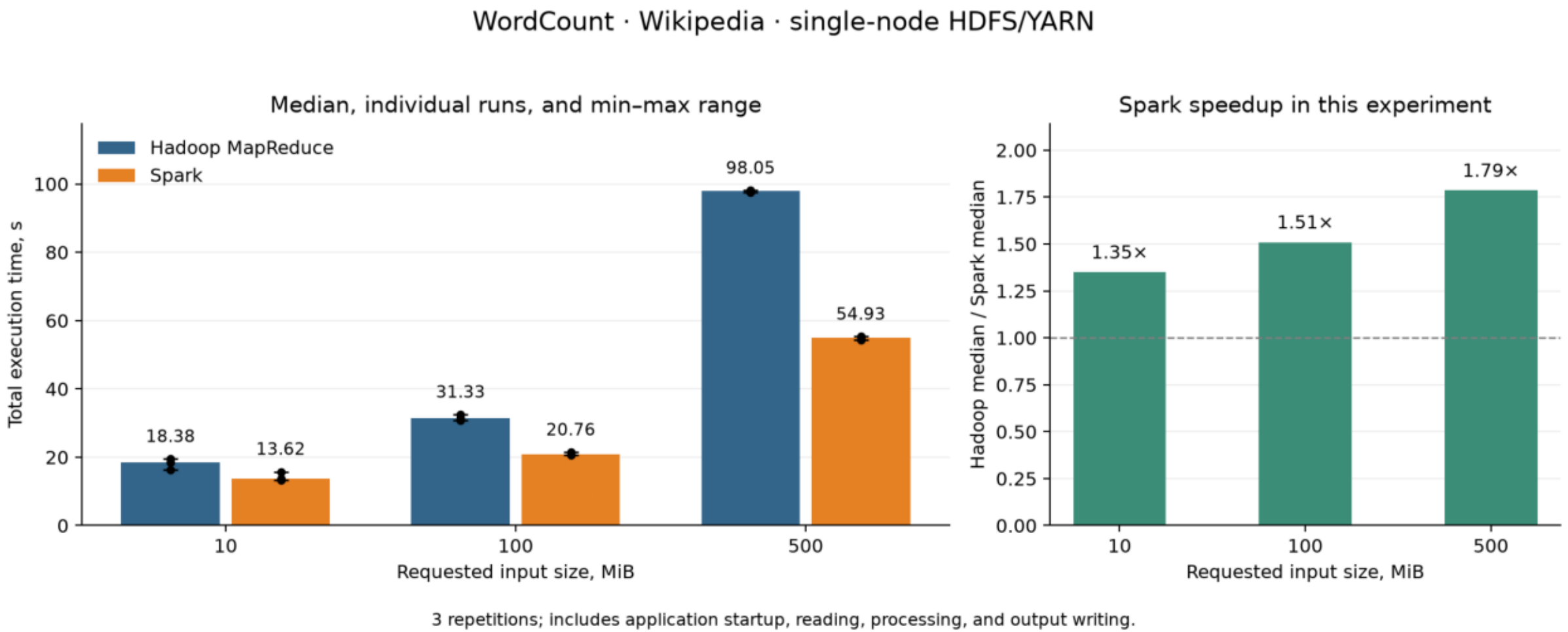
Recorded manifests; config/wordcount.json and config/kmeans.json

Study design: WordCount

- Wikipedia prefixes: 10, 100, and 500 MiB; three repeats per engine.
- One shared tokenizer: lowercase Unicode letters/digits.
- Count every token and save the complete dictionary to HDFS.
- Timer includes startup, reading, shuffle, processing, and writing.
- Input preparation/upload and output verification are outside timing.
- 18 applications; engine order alternated; no persistence or cache reset.

Own implementation: hadoop/mapper.py, reducer.py; spark/wordcount.py

WordCount results



All complete outputs matched. Median speedup: 1.35× → 1.79× in this configuration.

Own implementation: text → shared vectors

- 1000 JSONL articles preserve IDs, titles, URLs, and document boundaries.
- TF-IDF: English stop words; min_df=2; max_df=0.9; 1000 features.
- Sublinear TF, smoothed IDF, and L2 normalization; no zero vectors.
- One sparse vector file is shared by Hadoop and Spark.
- 5 distinct initial centers; seed 42; preprocessing excluded from timing.

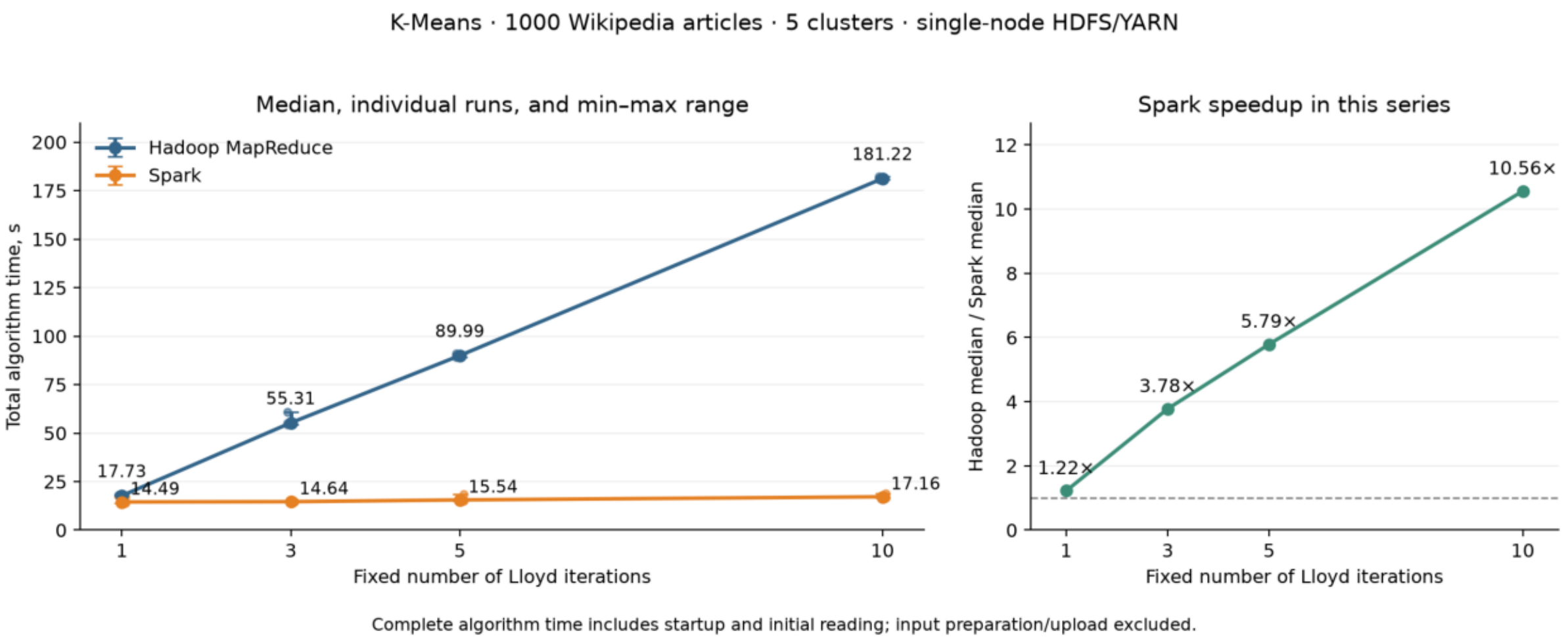
scripts/prepare_vectors.py; config/tfidf.json; initial_centers.json

Study design: iterative Lloyd K-Means

- Assign documents to nearest centers, then recompute arithmetic means.
- Same initial centers; exact ties use lowest index; empty clusters keep centers.
- Fixed 1, 3, 5, and 10 iterations; no early stopping; three repeats.
- Hadoop: new job and center transfer each iteration; no combiner.
- Spark: materialize MEMORY_AND_DISK once; aggregate partition sums.
- 24 applications; total time includes initial loading and startup.

Own Hadoop/Spark runners; common/kmeans.py

K-Means results



At 10 iterations: 181.219 s vs 17.158 s; median ratio 10.56×. Three repeats per condition.

Own implementation and recorded experiments

Own implementation: WordCount source code

hadoop/mapper.py + reducer.py

```
 11  def main():
 12      for line in sys.stdin:
 13          for word in tokenize(line):
 14              print(f"{word}\t1")
 15

# hadoop/reducer.py
  5  def main():
  6      previous = None
  7      total = 0
  8      for line in sys.stdin:
  9          word, count = line.rstrip("\n").split("\t")
 10          if previous is not None and word != previous:
 11              print(f"{previous}\t{total}")
 12              total = 0
 13          previous = word
 14          total += int(count)
 15      if previous is not None:
 16          print(f"{previous}\t{total}")
 17
```

spark/wordcount.py

```
 24  lines = sc.textFile(args.input,
         minPartitions=args.partitions)
 25  print(f'Input partitions: {lines.getNumPartitions()};
         output partitions: {args.partitions}',
 26        flush=True)
 27  counts = (lines
 28            .flatMap(tokenize).map(lambda word: (word, 1))
 29            .reduceByKey(add, numPartitions=args.partitions))
 30  counts.map(lambda pair:
         f"{pair[0]}\t{pair[1]}").saveAsTextFile(args.output)
```

Our Python counts words; Hadoop performs shuffle/sort. Spark expresses the same task through its RDD API.

Verbatim source excerpts with line numbers; long lines wrapped

Own implementation: Hadoop K-Means

hadoop/kmeans_mapper.py

```
16  norms = [math.fsum(value * value for value in center) for
        center in centers]
17  for line_number, line in enumerate(lines, 1):
18      try:
19          vector = validate_vector(json.loads(line),
        len(centers[0]))
20          cluster, _ = nearest_center(vector, centers, norms)
21      except (ValueError, TypeError) as error:
22          raise ValueError(f'Input line {line_number}:
        {error}') from error
23      payload = {key: vector[key] for key in ('id', 'size',
        'indices', 'values')}
24      # Only the cluster is the Hadoop key; the complete
        vector is its value.
25      output.write(f'{cluster}\t{json.dumps(payload,
        ensure_ascii=False, allow_nan=False)}\n')
26
```

hadoop/kmeans_reducer.py

```
19  def emit():
20      center = update_centers([centers[previous_cluster]],
        [total], [count])[0]
21      payload = {'count': count, 'center': center}
22      output.write(f'{previous_cluster}\t{json.dumps(payload,
         allow_nan=False)}\n')
    ...
38  if total is None:
39      total = [0.0] * len(centers[cluster])
40  previous_key, previous_cluster = key, cluster
41  for index, value in zip(vector['indices'],
        vector['values']):
42      total[index] += value
43  count += 1
```

Assign to a cluster → Hadoop groups vectors → reducer calculates the coordinate mean.

Actual mapper/reducer functions; full files are included in the project

Own implementation: persisted Spark K-Means

spark/kmeans_job.py

```
36  vectors = (sc.textFile(args.input,
        minPartitions=args.partitions)
37              .map(lambda line:
        validate_vector(json.loads(line), dimension))
38              .persist(StorageLevel.MEMORY_AND_DISK))
39  count = vectors.count()  # Materialize the cache; this time
         is included.

46  broadcast = sc.broadcast(centers)
47  try:
48      partials = (vectors.mapPartitions(lambda items:
        partition_sums(items, broadcast.value))
49                  .reduceByKey(combine_sums,
        numPartitions=args.partitions).collect())
50  finally:
51      broadcast.destroy(blocking=True)
```

What this code actually does

- Parse, validate, and persist input vectors.
- count() includes cache materialization.
- Broadcast current centers to workers.
- Combine sums/counts with reduceByKey.
- Collect only small center aggregates.
- Release the broadcast after each action.

One application, fixed iterations; initial loading/persistence is included, not an untimed warm-up.

Own implementation: execution and verification

scripts/check_kmeans.py

```
24  if states[0]['counts'] != states[1]['counts']:
25      raise ValueError(f'Cluster counts differ at iteration
        {iteration}')
    ...
30  for a, b in zip(states[0]['centers'],
        states[1]['centers']):
31      for x, y in zip(a, b):
32          if not math.isfinite(x) or not math.isfinite(y) or
        not math.isclose(x, y, rel_tol=rel_tol,
        abs_tol=abs_tol):
33              raise ValueError(f'Center coordinates differ at
         iteration {iteration}')
34          largest_difference = max(largest_difference, abs(x
        - y))
```

Executable evidence

```
python scripts/run_hadoop_kmeans.py --iterations 2

python scripts/run_spark_kmeans.py --iterations 2

python scripts/check_kmeans.py
  <hadoop-output> <spark-output>
```

Retained for every run:

- manifest.json + source/input hashes
- Per-iteration centers and counts
- Engine logs, timings, and _SUCCESS

34 automated checks passed. All 18 WordCount and 24 K-Means applications completed and were validated.

Recorded artifacts, source commands, and per-iteration numerical checks

Correctness, interpretation, and limitations

- All K-Means iteration centers/counts agree; coordinate differences ~7e-17.
- Objective decreases: 1346.436 initially → 722.709 after ten iterations.
- Repeated jobs grow Hadoop time; Spark reuses its application and vectors.
- Not a cache-only test: aggregation, startup, shuffle, and scheduling differ.
- Small corpus; one node; three repeats; no RAM/CPU profiling or quality labels.
- WordCount ranking agrees with the paper; its size-scaling trend does not.

Conclusions and references

- Two paper-inspired experiments completed using real distributed frameworks.
- WordCount median speedup: 1.35–1.79×; K-Means: 1.22–10.56×.
- Results validated; source code, commands, manifests, and logs retained.
- The iterative advantage is clear for this workload, not a universal guarantee.
- Next: larger samples, more repeats, local Hadoop aggregation, and ablations.

Ketu, Mishra & Agarwal (2020), Computación y Sistemas 24(2), 669–686.
DOI: 10.13053/CyS-24-2-3401 · Apache Hadoop/Spark docs · scikit-learn docs